Agents SDK: sessioni e conversazione

Modulo 3 · Lezione 11



Demo: un follow-up senza storia

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py senza` · due run, un minuto

si torna alle slide subito dopo

L'ASSISTENTE DELLA LEZIONE SCORSA · IL DIALOGO 1 DI MODULO 3 · LEZIONE 3

1 · «Perché il piano si ferma sempre a metà corsa?»

2 · «E per il secondo caso?»

Cosa guardare: il secondo run. Quale tool chiama, e con quale argomento.



Ogni run parte da zero

```
Tu> Perché il piano si ferma sempre a metà corsa?
tool cerca_documenti({"domanda":"Perché il piano della scrivania LumaDesk si
ferma sempre a metà corsa? Cause e soluzione."})
2 chiamate · 1.562 token in
→ tre cause: un ostacolo [ticket-supporto-001], una multipresa sovraccarica
[ticket-supporto-005], la calibrazione [manuale-lumadesk-004]

Tu> E per il secondo caso?
nessun tool
1 chiamata · 552 token in
→ Non ho abbastanza contesto per capire a quale “secondo caso” ti riferisci.
```

cosa vede il modello al secondo run

· le istruzioni

· i due tool

· «E per il secondo caso?»

Nient'altro: la prima domanda e la sua risposta non ci sono.

> Non cerca «secondo caso»: non cerca proprio, e chiede. È il problema di Modulo 3 · Lezione 3. Stasera la storia la tiene una **sessione** dell'SDK.



Agenda

	Blocco	Dove	Tempo
	Apertura: un follow-up senza storia	slide · `scaffolding/demo.py`	5′
	Setup: venv, chiave, collection	`scripts/`	5′
	Teoria: la sessione, cosa entra nella storia, la domanda riscritta dal modello, il tetto · una dimostrazione	slide · `scaffolding/demo.py`	25′
	Codice spiegato: cosa cambia nel file, poi il lancio	`working/assistente_sdk.py`	15′
	Esercizio: la sessione su file (TODO 1), il tetto alla storia (TODO 2) · poi 5′ di rientro	`working/assistente_sdk.py`	55′ + 5′
	Recap e prossima lezione	slide	10′

Un file solo, `working/assistente_sdk.py`, nella cartella `scripts/` di questa lezione. Le istruzioni dell'esercizio sono in `CONSEGNA.md`.



Setup

FATE VOI — ognuno sulla propria macchina

`la cartella scripts/ di questa lezione · 5 minuti · istruzioni in CONSEGNA.md`

poi si torna alle slide:
l'installazione finisce da sola

I COMANDI

```
cd scripts
python3 -m venv .venv && source .venv/bin/activate
Windows: python -m venv .venv e .venv\Scripts\activate
pip install -r requirements.txt
python -c "import agents; print('ok')"  quando pip ha finito
cp ../../l10_agents-sdk-loop-gestito/scripts/.env .env
cp -r ../../l10_agents-sdk-loop-gestito/scripts/chroma .
# la seconda solo se hai chroma/ nella lezione scorsa
```

`requirements.txt` è **uguale** a quello della lezione scorsa: la sessione è già dentro `openai-agents`. Chi ha ancora quel venv può attivare quello e non installare niente: il comando è in `CONSEGNA.md`.

Il `.env` è quello della lezione scorsa; se non ce l'hai più, alza la mano. La collection: se non copi `chroma/`, il primo lancio la ricostruisce da solo, due minuti. I path sono quelli della repo del corso (`modulo3-rag/l11_…/scripts`); i comandi per Windows sono in `CONSEGNA.md`.

L'assistente non si lancia adesso: il primo lancio è nel codice spiegato.



Cosa fa una sessione

`run 1`
«Perché il piano si ferma…?» → il giro → la risposta

→
dopo il run: salva gli item

`la sessione`
la domanda · le `function_call` · i loro output · la risposta

→
prima del run: li mette davanti alla domanda

`run 2`
la storia + «E per il secondo caso?» → il giro → la risposta

```
sessione = SQLiteSession("mario") # in memoria
sessione = SQLiteSession("mario", SESSIONI) # su file: SESSIONI è scripts/sessioni.db
result = Runner.run_sync(agente, domanda, session=sessione)
```

In memoria: la storia si perde quando il programma si chiude.

Su file: lo stesso nome ritrova la stessa conversazione, anche dopo un riavvio.

L'agente non cambia: la storia non sta nell'`Agent`, sta nella sessione.



Demo: cosa c’è nella sessione

DEMO DEL DOCENTE — guardate lo schermo

`python scaffolding/demo.py storia · due run, un minuto`

si torna alle slide
subito dopo

GLI STESSI DUE SCAMBI, CON UNA SESSIONE IN MEMORIA · COSA GUARDARE

1 · il secondo run: cerca con una domanda riscritta, o risponde senza tool

2 · gli item nella sessione, nell'ordine; e quanto è lungo l'output di `cerca_documenti`



Cosa si è visto: gli item nella sessione

```
scambio 1
message (user) Perché il piano si ferma sempre a metà corsa?
function_call cerca_documenti({"domanda": …})
function_call_output [ticket-supporto-001] … ≈ 400 token
message (assistant) Le cause più comuni sono: …

scambio 2
message (user) E per il secondo caso?
function_call cerca_documenti({"domanda":"Secondo caso: multipresa
sovraccarica …"})
function_call_output [ticket-supporto-005] …
message (assistant) Collega la scrivania a una presa dedicata …
```

Otto item, quattro per scambio: i tipi della traccia della lezione scorsa, salvati (`function_call` = `ToolCallItem`, `function_call_output` = `ToolCallOutputItem`). In più la domanda, che in `new_items` non c'era.

Un output di `cerca_documenti` sono i **quattro passaggi**: circa 400 token.

Se il secondo scambio risponde senza tool, gli item sono sei.

> Al terzo run vanno tutti davanti alla domanda nuova.



Turno e scambio

```
Tu> E per il secondo caso?
```

`turno 1 · il modello chiede cerca_documenti` → l'SDK esegue il tool → `turno 2 · il modello risponde`

```
→ Collega la scrivania a una presa dedicata … [ticket-supporto-005]
```

turno

Quello della lezione scorsa: **una chiamata al modello.** `max_turns` conta i turni di un run, cioè di uno scambio.

scambio

Una domanda della chat e la sua risposta. Di solito due turni; uno se risponde senza tool.

Il dialogo lungo dell'esercizio: sette scambi, quattordici turni. A Modulo 3 · Lezione 3 «turno» voleva dire scambio.



La corrispondenza con chat.py

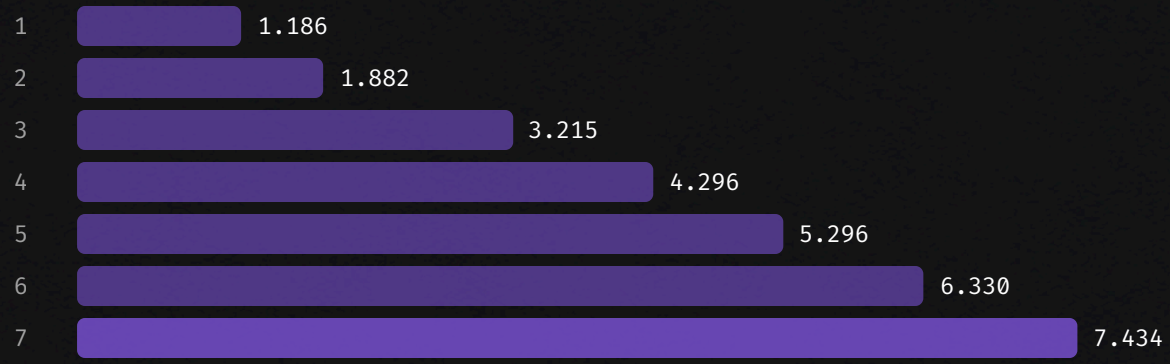
Modulo 3 · Lezione 3 (chat.py)		stasera (SDK)
`la lista storia`	→	`la sessione`
`salva_sessione / carica_sessione, su JSON`	→	`SQLiteSession(nome, file)`
`tronca(storia)`: il buffer, per budget di token	→	`SessionSettings(limit=N)`: gli ultimi N item
`condensa(storia, domanda)`: una chiamata in più	→	l'argomento di `cerca_documenti`: lo scriveva già il modello, ora con la storia davanti
`riassumi(storia)`	→	nella SQLiteSession non c'è: il tetto taglia, non riassume

Una differenza: nella sessione entrano **anche i passaggi dei documenti**. A Modulo 3 · Lezione 3 in storia andavano solo domande e risposte.



Cosa entra nella storia: tutto

Il dialogo lungo di Modulo 3 · Lezione 3, con la sessione: token in input per scambio



+ circa 1.000 token a scambio

ogni scambio aggiunge ≈500 token alla storia, quasi tutti passaggi; ogni turno rilegge tutta la storia, e uno scambio sono due turni

senza sessione: **550–1.700**

una domanda sola e i suoi passaggi

Il costo è piccolo (circa $0,007 il dialogo intero), ma cresce a ogni scambio.

La domanda per il retrieval la scrive il modello

A Modulo 3 · Lezione 3 la riscriveva `condensa()`, sempre, con una chiamata in più. Qui è l'argomento di `cerca_documenti`: il modello lo scrive con la storia davanti.

follow-up	senza sessione	con la sessione
«E dall'estero?» dopo: quanti giorni da remoto?	`È possibile acquistare o ricevere ordini dall'estero? Spedizioni internazionali…`	`Quanto posso lavorare da remoto dall'estero e quali autorizzazioni servono?`
«In quel caso come mi viene rimborsata?» dopo: comprata con un codice promozionale	`Come viene effettuato il rimborso in quel caso? …` risponde sicuro col rimborso standard: sbagliato per un acquisto in promozione	`Per un prodotto acquistato con un codice promozionale, come viene effettuato il rimborso in caso di reso?`
«Perché no?» dopo: il braccio LMX-4400 sul piano LD-115	`nessun tool: «non ho abbastanza contesto»`	`Perché il braccio LMX-4400 non è compatibile con il piano fisso LD-115?`

La domanda riscritta si legge nella traccia, nella riga del tool.



I follow-up di Modulo 3 · Lezione 3, misurati

I sette dialoghi, tre domande ciascuno (due follow-up), due prove: ventotto follow-up per parte. Giusto = la risposta ha il fatto giusto dei documenti.

senza sessione

14 su 28

7 su 14 in ogni prova.

Passano i follow-up che contengono già la parola da cercare («E sulla centralina?»), o per fortuna.

I sette che falliscono:

· tre volte **nessun tool**: il modello chiede di cosa si parla

· quattro volte un **argomento scritto senza contesto**: «non ho trovato», o una risposta plausibile e sbagliata (il rimborso standard; in una prova la portata della scrivania invece del braccio)

con la sessione

27 su 28

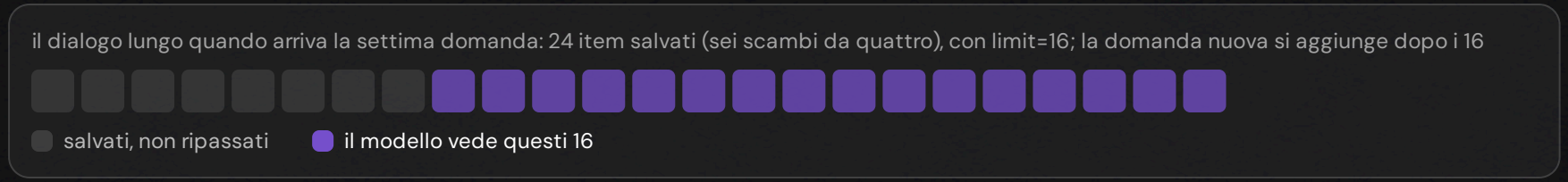
· l'unico errore viene dal primo scambio, che aveva mancato il documento: il follow-up l'ha ereditato

· a volte **nessun tool**: la risposta è già nella storia, e cita il passaggio di prima



Il tetto alla storia: SessionSettings(limit=N)

La sessione salva tutto; al modello ripassa solo gli ultimi N item. È il buffer di Modulo 3 · Lezione 3, contato in item invece che in token: un item può essere una domanda di una riga o quattro passaggi.




```
SQLiteSession(nome, file, session_settings=SessionSettings(limit=16))
```

Il taglio conta item, non scambi. Se cade a metà di uno scambio, resta l'output di un tool senza la sua `function_call`, uscita dal tetto: l'SDK lo toglie prima di chiamare il modello, nessun errore.

> Quello che esce dal tetto il modello non lo vede più. Con quale N, e cosa si perde: è il TODO 2.

Il file: working/assistente_sdk.py

```
MODELLO — gpt-5.6-luna · K — 4 · MAX_TURNI — 10
SESSIONI — il file delle sessioni, scripts/sessioni.db · LIMITE — il tetto alla storia, None = tutto
cerca_documenti · query_db — i due tool, già scritti
ISTRUZIONI · agente = Agent(…) — uguali alla lezione scorsa
strada(result) — già scritta: sotto ogni risposta dice se ha cercato, e dove
TODO 1 e TODO 2 · apri_sessione() — la sessione su file, con un nome, e il tetto
stampa_traccia(result)
main — la chat: Tu> … e Runner.run_sync(agente, domanda, session=sessione, …)
```

È la soluzione della lezione scorsa, con la sessione in più: chi non aveva finito i TODO riparte da qui con tutto scritto. In viola le righe nuove.



Codice spiegato: cosa cambia nel file

CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file

`working/assistente_sdk.py` · tutto già scritto tranne i TODO · 15 minuti

si rientra dopo il lancio, con l'output a schermo

IL DOCENTE

Cosa cambia rispetto alla lezione scorsa, dall'alto in basso. Poi il lancio, con le due domande dell'apertura (sono anche in `CONSEGNA.md`):

```
python working/assistente_sdk.py
```

Poi Invio a vuoto per uscire, un nuovo lancio, e di nuovo «E per il secondo caso?».

VOI

Il vostro `working/assistente_sdk.py` aperto. Dopo il docente, lanciate anche voi e rifate le stesse due domande, poi il riavvio. Al primo lancio senza `chroma/` indicizza, due minuti.



Cosa cambia rispetto alla lezione scorsa

la lezione scorsa		stasera
`from agents import …`	→	in più SQLiteSession, SessionSettings
`MODELLO, K, MAX_TURNI`	→	in più SESSIONI e LIMITE
`main: le quattro DOMANDE e la tabella`	→	una chat: input("Tu> "), run_sync(…, session=sessione), traccia e strada sotto ogni risposta
—	→	apri_sessione(), il TODO; finché manca, SQLiteSession("aula"), in memoria
`tool, ISTRUZIONI, agente, strada()`	=	uguali
`scaffolding/aikit/`	→	rag, rerank e tools con Luna come modello di default (stasera non si usano)

IL LANCIO

Il follow-up passa. Nella traccia, o il tool con la domanda riscritta, o nessun tool e la risposta presa dalla storia.

DOPO IL RIAVVIO

La sessione «aula» era in memoria: è vuota. «E per il secondo caso?» torna a «non ho abbastanza contesto». È il TODO 1.



Esercizio: la sessione su file, il tetto

SCRIVETE VOI — ognuno nel proprio file

`working/assistente_sdk.py` · TODO 1 e TODO 2 · 55 minuti · istruzioni e domande in `CONSEGNA.md`

poi 5' di rientro sulle slide, con la soluzione

`TODO 1 · la sessione su file (25')`

`apri_sessione()` chiede il nome con `input()` e apre la sessione sul file `SESSIONI`. Prova: «Il braccio LMX-4400 è compatibile con il piano fisso LD-115?», Invio a vuoto, rilancio con lo stesso nome, «Perché no?».

`FATTO QUANDO` · dopo il riavvio «Perché no?» risponde col **motivo**: il piano è troppo sottile per il morsetto (senza storia: «non ho abbastanza contesto»)

`TODO 2 · il tetto alla storia (30')`

Nella stessa riga: `session_settings=SessionSettings(limit=LIMITE)`. Prova: il dialogo lungo, sette domande in `CONSEGNA.md`, una volta con `LIMITE = None` e una con `LIMITE = 16`, ogni volta con un nome di sessione nuovo.

`FATTO QUANDO` · con `None` l'ultima domanda trova il peso (34 kg); con 16 i token dell'ultimo scambio sono più bassi e la scrivania del primo scambio non si trova più

Se finisci prima: il dialogo in fondo a `CONSEGNA.md`, poi il suo follow-up da solo in una sessione nuova, leggendo nella traccia la domanda scritta dal modello; poi `LIMITE = 24` e `LIMITE = 10`. Altri dialoghi li ha il docente.



Rientro: la soluzione e i numeri

```python
def apri_sessione():
    nome = input("Nome della sessione: ").strip()
    return SQLiteSession(nome, SESSIONI, session_settings=SessionSettings(limit=LIMITE))
```

`LIMITE = None` · quattro prove

L'ultimo scambio sta tra **7.262 e 8.146** token in. «La LumaDesk LD-200 pesa 34 kg», quattro volte su quattro.

`LIMITE = 16` · quattro prove

Resta sotto i **6.000**. Quattro volte su quattro il modello non sa più quale scrivania: tre volte dice che non trova il peso, una volta tira a indovinare tra LD-200 e LD-210.

Senza file la sessione è in memoria.

Lo stesso nome riprende; un nome nuovo parte da zero.

> Chi non ha finito lo ricopia adesso.



Recap

1. Una **sessione** salva gli item di ogni run e li ripassa al run dopo. Con un file e un nome, la conversazione riprende anche dopo un riavvio.

2. Nella storia entra **tutto**, anche i passaggi dei documenti: circa 500 token di storia a scambio, cioè circa mille token in input in più.

3. La domanda per il retrieval la **scrive il modello**, con la storia davanti: sui dialoghi di Modulo 3 · Lezione 3, 27 follow-up giusti su 28, contro 14 su 28 senza sessione.

4. Il **tetto** (`limit`, in item) ferma la crescita della storia, e quello che esce il modello non lo vede più.



La prossima lezione

stasera

la sessione, la storia, il tetto, sul file della lezione

→

Modulo 3 · Lezione 12: il vostro assistente sull'SDK

portate il vostro assistente.py di Modulo 3 · Lezione 9: in aula lo passate sull'SDK, sessione compresa, e lo misurate sul query set del modulo, come a Modulo 3 · Lezione 6, controllando anche la strada presa



Grazie

prossima lezione: Modulo 3 · Lezione 12

